Atividade em Casa — CRUD de Projetos (NestJS)

Programação para Web 2 · Aula 05 (NestJS e validação de formulários)

> Entrega individual. Continue no MESMO backend que você construiu na aula.

Objetivo

Reforçar, por conta própria, tudo o que vimos hoje no NestJS — **módulo, controller, service, DTO**, **injeção de dependência** e **validação** com `class-validator` + `ValidationPipe` — criando um **novo recurso CRUD**: **Projetos**.

Contexto

Hoje criamos o CRUD de **Tarefas**. Agora você vai criar, seguindo o **mesmo padrão**, o CRUD de **Projetos**, do zero, dentro do mesmo backend NestJS. A ideia é que um "projeto" possa, no futuro, agrupar tarefas (isso é um desafio opcional no fim).

O recurso: Projeto

Campo	Tipo	Regras de validação
`id`	number	gerado pelo backend
`nome`	string	**obrigatório**, de 3 a 60 caracteres
`descricao`	string	**opcional**, até 200 caracteres
`cor`	string	**obrigatório**, um de: `vermelho`, `verde`, `azul`, `amarelo`, `roxo`

Endpoints (em `/api/projetos`)

Método e rota	O que faz	Status de sucesso
`GET /api/projetos`	lista os projetos	200
`GET /api/projetos/:id`	um projeto (404 se não existir)	200
`POST /api/projetos`	cria (valida o body)	201
`PUT /api/projetos/:id`	edita (valida o body)	200
`DELETE /api/projetos/:id`	exclui	204

Passo a passo

1) Gerar o recurso com o Nest CLI (recomendado):

```
cd backend-inicial        # o backend que você construiu na aula
npx nest generate resource projetos
# Perguntas do CLI:
#   - Transport layer:  REST API
#   - Generate CRUD entry points?  Yes
```

Isso cria o módulo, o controller, o service, os DTOs e a entity já ligados — e registra o `ProjetosModule` no `AppModule` automaticamente.

> Alternativa manual: copie a pasta `src/tarefas/` para `src/projetos/`, renomeie as classes e adapte. Não esqueça de importar o `ProjetosModule` no `AppModule`.

2) Entidade: defina `Projeto` (`id`, `nome`, `descricao?`, `cor`) em `entities/projeto.entity.ts`.

3) Service: crie um array em memória (com 1 ou 2 projetos de exemplo) e implemente `findAll`, `findOne` (com `NotFoundException` no 404), `create`, `update` e `remove` — igualzinho ao de tarefas.

4) Controller: implemente as 5 rotas (`@Get`, `@Get(':id')` com `ParseIntPipe`, `@Post`, `@Put`, `@Delete` com `@HttpCode(204)`), injetando o service pelo construtor.

5) DTOs + validação:

- `CreateProjetoDto`:
  - `nome`: `@IsString`, `@IsNotEmpty`, `@MinLength(3)`, `@MaxLength(60)`
  - `descricao`: `@IsOptional`, `@IsString`, `@MaxLength(200)`
  - `cor`: `@IsIn(['vermelho','verde','azul','amarelo','roxo'])`
- `UpdateProjetoDto extends PartialType(CreateProjetoDto)`

> O `ValidationPipe` já está global (ligamos em `main.ts`), então ele passa a validar os DTOs de projetos **sem nenhuma configuração extra**.

Dica: a `cor` usa um validador novo — `@IsIn`. Dê uma olhada também no `@IsEnum` na documentação do `class-validator`.

Teste no Postman (critérios de aceite)

- [ ] `GET /api/projetos` -> **200** com a lista
- [ ] `POST /api/projetos` com `{ "nome": "Faculdade", "cor": "azul" }` -> **201**
- [ ] `POST` com `{ "nome": "ab", "cor": "azul" }` -> **400** (mínimo de 3 caracteres)
- [ ] `POST` com `{ "nome": "Trabalho", "cor": "rosa" }` -> **400** (cor inválida)
- [ ] `POST` com um campo a mais (ex.: `"xpto": 1`) -> **400** (`forbidNonWhitelisted`)
- [ ] `GET /api/projetos/999` -> **404**
- [ ] `PUT /api/projetos/1` com `{ "nome": "Faculdade 2026" }` -> **200**
- [ ] `DELETE /api/projetos/1` -> **204**

Desafios (opcionais)

- **Relacionar tarefas e projetos:** adicione `projetoId?` (number) ao `CreateTarefaDto` validando com `@IsOptional()` e `@IsInt()`. Depois, permita filtrar as tarefas por projeto: `GET /api/tarefas?projetoId=1` (use `@Query`).
- **Frontend:** crie uma tela para listar e criar projetos, espelhando o que já existe para tarefas.

- **Mensagens personalizadas:** use a opção `{ message: '...' }` em cada validador para deixar os erros mais claros (como fizemos em tarefas).

Entrega

1. A pasta `src/projetos/` com todos os arquivos (module, controller, service, dto e entity).
2. **Prints do Postman** mostrando pelo menos os status **201, 400, 404 e 204**.
3. Se fez algum desafio, descreva o que mudou.

Rode com `npm run start:dev` e teste no Postman. Lembre: os dados estão **em memória** e somem quando o servidor reinicia.